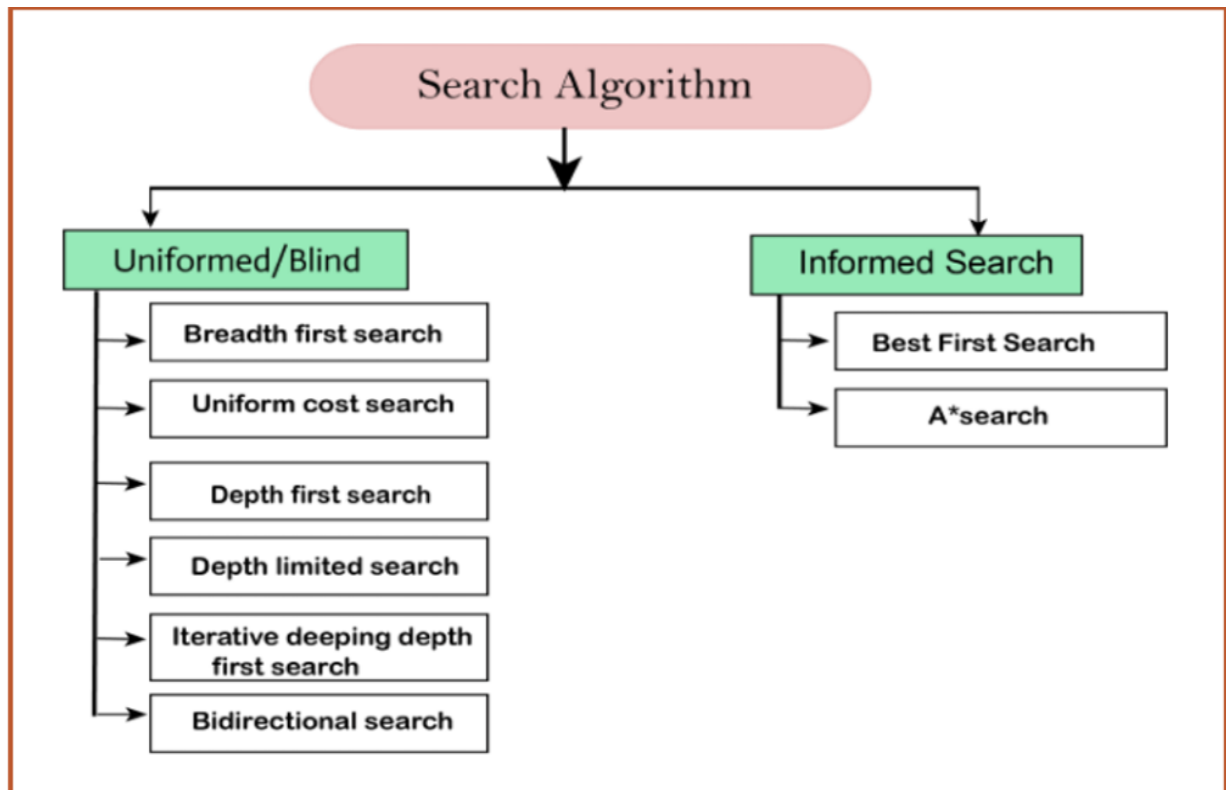


UNIT -3 Problem solving



UNINFORMED SEARCH ALGORITHMS

Uninformed search is a class of general-purpose search algorithms which **operates in brute force-way**.

Uninformed search algorithms **do not have additional information** about state or search space other than how to traverse the tree, so it is **also called blind search**.

Following are the **various types of uninformed** search algorithms:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first search

1. BREADTH-FIRST SEARCH:

- Breadth-first search is the most common search strategy for traversing a tree or graph.
- This algorithm **searches breadthwise in a tree or graph**, so it is called breadth-first search.
- BFS algorithm **starts searching from the root node** of the tree and **expands all successor node** at the current level before moving to nodes of next level.
- It is a graph/tree search algorithm that **explores nodes level by level** starting from the root node.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using **FIFO queue data structure**.

Advantages:

- BFS will provide a solution, if any solution exists.
- If there are **more than one solution** for a given problem, then **BFS will provide the minimal solution** which requires the least number of steps.

Disadvantages:

- It **requires lots of memory** since each level of the tree must be saved into memory to expand the next level.
- BFS **needs lots of time** if the solution is far away from the root node.

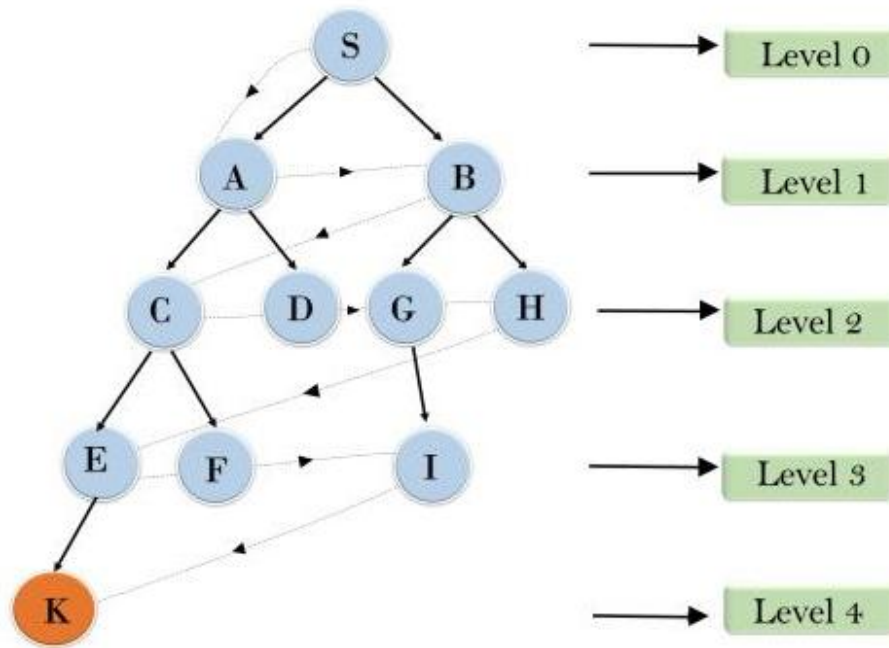
Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm **from the root node S to goal node K**.

BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B---->C--->D---->G--->H--->E---->F---->I---->K

Breadth First Search



Time Complexity:

Time Complexity of BFS algorithm can be **obtained by the number of nodes traversed** in BFS until the shallowest Node.

Where the **d**= **depth** of shallowest solution and **b** is a node at every state.

$$T(b) = 1 + b^1 + b^2 + \dots + b^d = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means **if the shallowest goal node is at some finite depth**, then **BFS will find a solution**.

Optimality: BFS is **optimal** if **path cost is a non-decreasing function** of the depth of the node.

- The **cost never decreases** as depth increases.
- Each **deeper level has equal or higher cost** than the previous level.

2. DEPTH-FIRST SEARCH

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it **starts from the root node and follows each path to its greatest depth node** before moving to the next path.
- It explores a tree or graph by **going as deep as possible** along one branch before backtracking.
- DFS uses a **stack (LIFO – Last In First Out) data structure** for its implementation.

Advantage:

DFS requires **very less memory** as it only needs to store a stack of the nodes on the path from root node to the current node.

It takes **less time to reach to the goal** node than BFS algorithm (if it traverses in the right path).

Disadvantage:

There is the possibility that many **states keep re-occurring, and there is no guarantee of finding the solution.**

DFS algorithm goes for deep down searching and sometime it **may go to the infinite loop.**

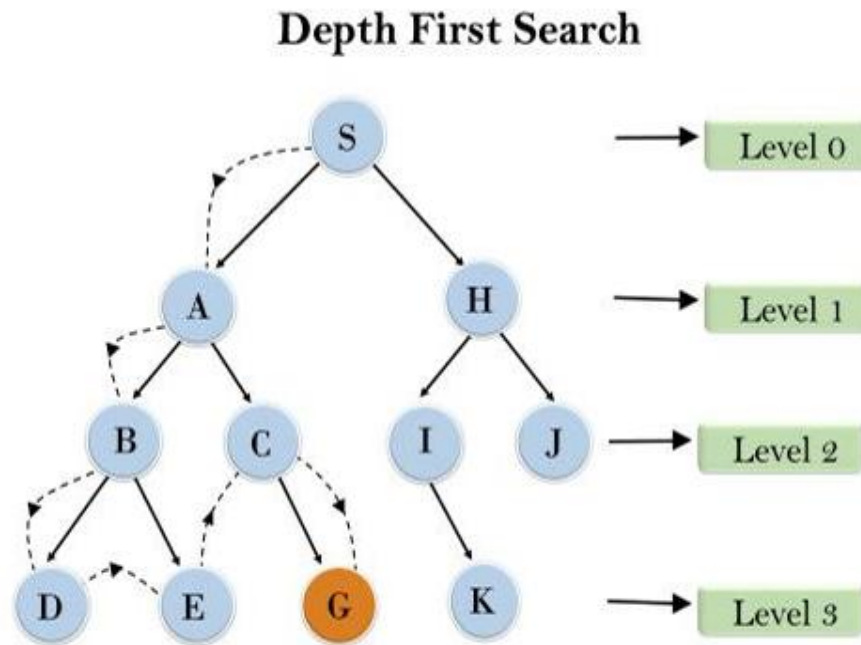
Example:

In the below search tree, we have shown the flow of depth-first search, and it will follow the order

as: Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found.

After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Completeness: DFS search algorithm is **complete within finite state space** as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \dots + n^m = O(n^m)$$

Where, **m = maximum depth** of any node and this can be much larger than **d** (Shallowest solution **depth**)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is $O(bm)$.

b = branching factor

maximum number of children each node can have.

3. DEPTH-LIMITED SEARCH ALGORITHM:

A depth-limited search algorithm is **similar to depth-first search with a predetermined limit.**

Depth-limited search **can solve the drawback of the infinite path in the Depth-first search.**

In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be **terminated with two Conditions of failure:**

Standard failure value: It indicates that problem does not have any solution.

Cut-off failure value: It defines no solution for the problem within a given depth limit.

Advantages:

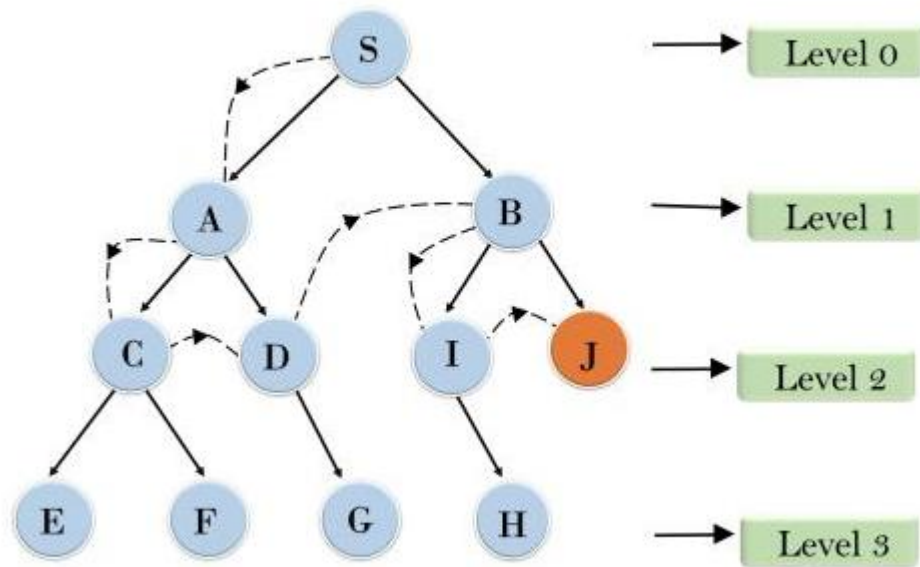
Depth-limited search is Memory efficient.

Disadvantages:

- o Depth-limited search also has a disadvantage of incompleteness.
- o It may not be optimal if the problem has more than one solution.

Example:

Depth Limited Search



Completeness: DLS search algorithm is complete if the solution is above the depth-limit.

Time Complexity: Time complexity of DLS algorithm is $O(b^\ell)$.

- b = branching factor (maximum number of children per node)
- ℓ = depth limit

Space Complexity: Space complexity of DLS algorithm is $O(b \times \ell)$.

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$.

d = depth of the shallowest goal node

(i.e., the level at which the first solution exists)

4. ITERATIVE DEEPENING DEPTH-FIRST SEARCH:

The iterative deepening algorithm is a combination of DFS and BFS algorithms.

This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm **performs depth-first search up to a certain "depth limit"**, and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the **benefits of Breadth-first search's fast search** and **depth first search's memory efficiency**.

The iterative search algorithm is **useful** uninformed search when **search space is large, and depth of goal node is unknown**.

Advantages:

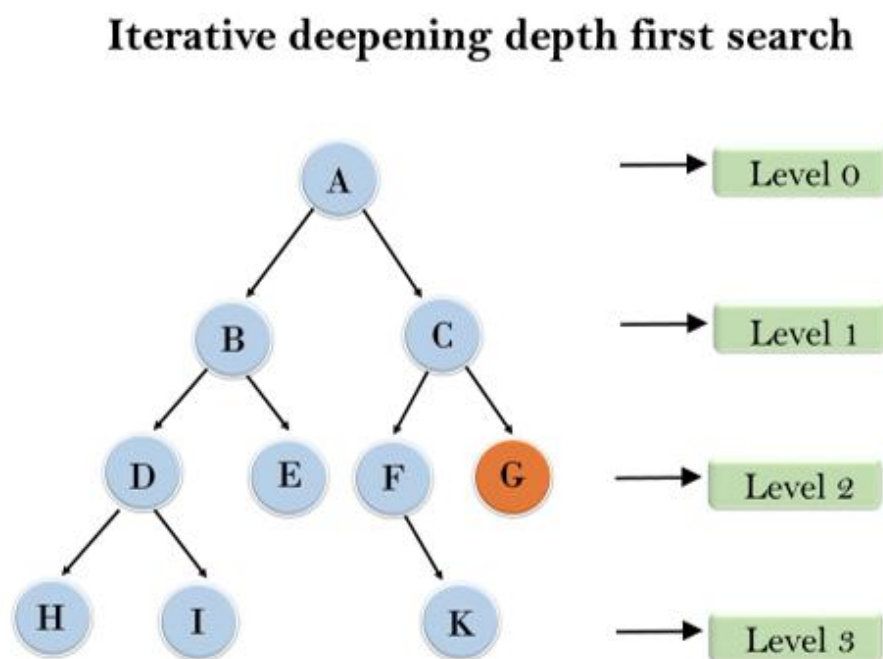
- It **combines the benefits of BFS and DFS** search algorithm in terms of fast search and memory efficiency.

Disadvantages:

- The main drawback of IDDFS is that **it repeats all the work of the previous phase**.

Example:

Following tree structure is showing the iterative deepening depth-first search. IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:



1'st	Iteration----->								A			
2'nd	Iteration----->		A,		B,		C					
3'rd	Iteration----->		A,	B,	D,	E,	C,	F,	G			
4'th	Iteration----->		A,	B,	D,	H,	I,	E,	C,	F,	K,	G

In the fourth iteration, the algorithm will find the goal node.

Completeness:

This algorithm is complete is if the branching factor is finite.

Time Complexity: Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity:

The space complexity of IDDFS will be $O(bd)$.

Optimal:

IDDFS algorithm is optimal if path cost is a **non- decreasing function** of the depth of the node.

- The **cost never decreases** as depth increases.
- Each **deeper level has equal or higher cost** than the previous level.

INFORMED SEARCH ALGORITHMS

Informed search algorithm **contains an array of knowledge** such as -

- how far we are from the goal
- path cost
- how to reach the goal node, etc.

This knowledge helps agents **explore less search space** and **find the goal node more efficiently**.

It is more useful for large search space.

It **uses the idea of heuristic**, so it is also **called Heuristic search**.

A **heuristic** is a **simple, practical method** used to make decisions or solve problems quickly—without needing a perfect or complete solution.

Heuristics function: Heuristic is a function which is used in Informed Search, and it **finds the most promising path**.

It **takes the current state** of the agent **as its input** and **produces the estimation** of how close agent is from the goal.

The heuristic method, however, **might not always give the best solution**, but it **guaranteed to find a good solution in reasonable time**.

Heuristic function estimates how close a state is to the goal.

It is **represented by $h(n)$** , and it **calculates the cost of an optimal path between the pair of states**.

The value of the heuristic function is always positive.

Admissibility of the **heuristic function** is given as:

$$h(n) \leq h^*(n)$$

Here, **$h(n)$** is **heuristic cost**, and **$h^*(n)$** is the **estimated cost**.

Hence **heuristic cost should be less than or equal to the estimated cost**.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms.

It **expands nodes** based on their **heuristic value $h(n)$** .

It **maintains two lists: OPEN and CLOSED**.

The **CLOSED list** contains **nodes** that have **already been expanded**, while the **OPEN list** contains **nodes** that have **not yet** been expanded.

In **each iteration**, the node n with the **lowest heuristic value** is selected, expanded, and its **successors are generated**. The node n is then moved to the CLOSED list.

The algorithm continues until a goal state is found.

In the informed search we will discuss **two main algorithms** which are given below:

- Best First Search Algorithm (Greedy search)
- A* Search Algorithm

1) **BEST-FIRST SEARCH ALGORITHM (GREEDY SEARCH):**

Greedy Best-First Search (a type of Best-First Search) is an informed search algorithm that **selects the node which appears to be closest to the goal** at a given moment.

It uses a **heuristic function** to guide the search.

- The evaluation function is:

$$f(n) = h(n)$$

Where:

- **$h(n)$** = estimated cost from node n to the goal

At each step, the algorithm:

- Expands the node with the **lowest heuristic value**
- Chooses the **most promising node** based on this estimate

This algorithm is typically implemented using a **priority queue**, where nodes with smaller **heuristic value $h(n)$** values are given higher priority.

Algorithm: Best-First Search

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, stop and return failure.

- **Step 3:** Remove the node n from the OPEN list that has the lowest value of $h(n)$, and place it in the CLOSED list.
- **Step 4:** Expand node n and generate its successor nodes.
- **Step 5:** Check each successor of node n . If any successor is a goal node, return success and terminate the search.
Otherwise, proceed to Step 6.
- **Step 6:** For each successor node, check whether it is already in the OPEN or CLOSED list. If it is not in either list, add it to the OPEN list.
- **Step 7:** Return to Step 2.

Advantages:

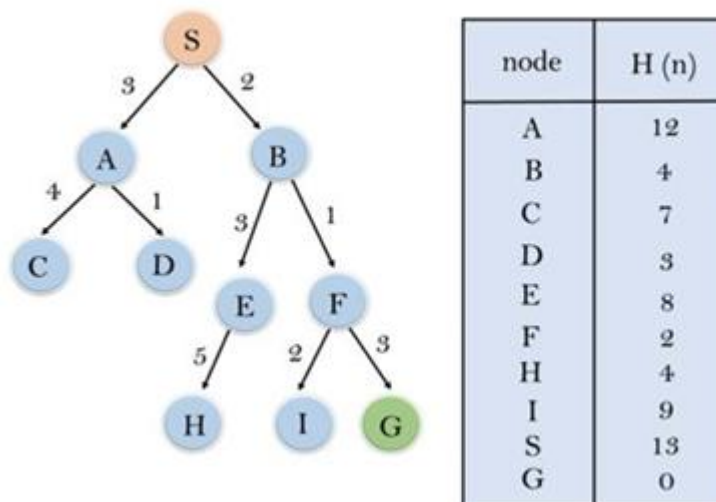
- **Best-First Search uses heuristic information**, which helps guide the search toward the goal more efficiently.
- It is generally **more efficient than BFS and DFS** when a good heuristic function $h(n)$ is available.
- It **expands fewer nodes** compared to uninformed search techniques.

Disadvantages:

- It can **get stuck in loops** if repeated states are not handled properly.
- It is **not optimal**, as it **does not guarantee the shortest path**.
- In the worst case, it may behave like **Depth-First Search**, exploring deep paths unnecessarily.
- Its performance **depends heavily on the quality of the heuristic function**.

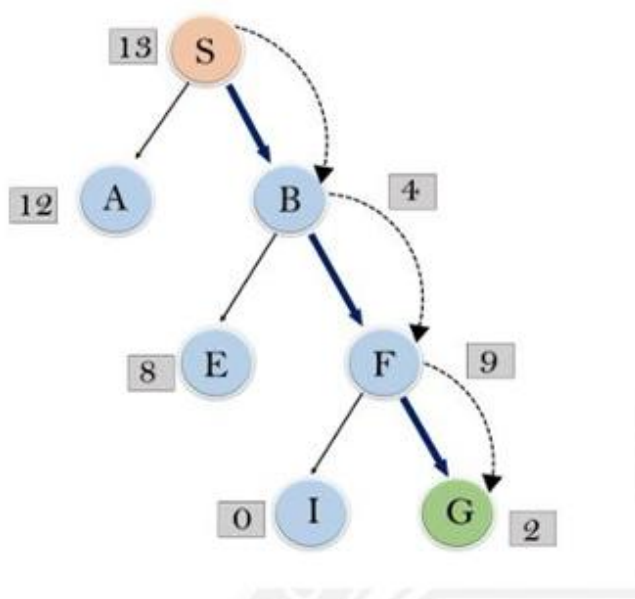
Example:

Consider the below search problem, and we will **traverse it using greedy best-first search**. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.



Solution -

In this search example, we are using two lists which are OPEN and CLOSED Lists. Following is the iteration for traversing the above example.



Initialization:

- OPEN = [S]
- CLOSED = []

After expanding S:

- OPEN = [B (4), A (12)]
- CLOSED = [S]

Iteration 1:

- Select **B**
- OPEN = [F (2), E (8), A (12)]
- CLOSED = [S, B]

Iteration 2:

- Select **F**
- OPEN = [G (0), E (8), I (9), A (12)]
- CLOSED = [S, B, F]

Iteration 3:

- Select **G (Goal)** → STOP

Final Path:

S → B → F → G

Time Complexity: The worst-case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst-case space complexity of Greedy best first search is $O(b^m)$.

Where, **m is the maximum depth** of the search space.

Complete: Greedy best-first search is also **incomplete**, even if the given state space is finite.

Optimal: Greedy best first search algorithm is **not optimal**.

2) A* SEARCH ALGORITHM:

A* search is one of the most **well-known forms of Best-First Search**.

It **uses a heuristic function $h(n)$** and the **cost to reach node n from the start state**, denoted as **$g(n)$** .

It **combines** the features of **Uniform Cost Search (UCS)** and **Greedy Best-First Search**, which allows it to solve problems efficiently.

- **Uniform Cost Search (UCS)** is a search algorithm that **expands the node with the lowest path cost** from the start node.
- It always selects the path with **minimum cumulative cost ($g(n)$)**.

A* search algorithm finds the shortest path in the search space using the heuristic function.

It **expands fewer nodes** and **provides an optimal solution** when the heuristic is admissible.

The **A* algorithm is similar to UCS**, **except** that it uses **$f(n) = g(n) + h(n)$** instead of only $g(n)$.

- **Here, $f(n)$** → estimated total cost
- **$g(n)$** → cost from start to node n
- **$h(n)$** → estimated cost from n to goal

Thus, **A* uses** both the **actual cost** and the **estimated cost** to reach the goal. The **sum** of these costs is called the **evaluation function $f(n)$** .

Algorithm of A* search:

Step 1: Place the starting node in the OPEN list. Initialize its **$g(n) = 0$** and compute **$f(n) = g(n) + h(n)$** .

Step 2: If the OPEN list is empty, return failure and stop.

Step 3: Select the node n from the OPEN list with the **smallest $f(n)$** .

- If n is the goal node, return success and terminate.

Step 4: Remove n from OPEN and place it in the CLOSED list. Expand n and generate all its successors.

Step 5: For each successor n' :

- Compute $g(n') = g(n) + \text{cost}(n, n')$ and $f(n') = g(n') + h(n')$.

$$g(n') = g(n) + \text{cost}(n, n')$$

This calculates the **actual cost to reach the new node (n')**

- $g(n) \rightarrow$ cost to reach current node n
- $\text{cost}(n, n') \rightarrow$ cost from n to its neighbor n'
- $g(n') =$ **total cost from start $\rightarrow n \rightarrow n'$**

$$f(n') = g(n') + h(n')$$

This calculates the **total estimated cost**

- $g(n') \rightarrow$ actual cost so far
- $h(n') \rightarrow$ estimated cost to reach goal

$f(n') =$ **total estimated cost from start to goal via n'**

- If n' is **not in OPEN or CLOSED**, add it to OPEN and set its parent to n .
- If n' is already in OPEN or CLOSED **with a higher cost**, update its $g(n')$, $f(n')$ and change its parent to n .

Step 6: Repeat from Step 2.

Advantages:

- A* search is one of the **most efficient search algorithms** when a good heuristic is used.
- It is **complete and optimal**, provided the heuristic function is admissible.
- It can solve **complex problems efficiently** by combining actual cost and heuristic information.

Disadvantages:

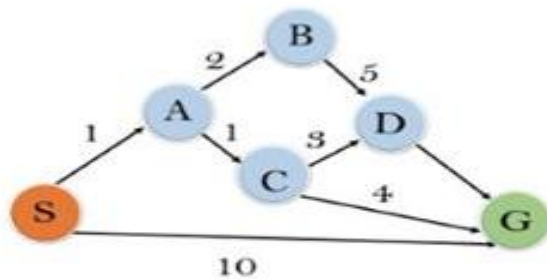
- A* has **high time and space complexity**, especially for large search spaces.
- It requires a **large amount of memory**, as it stores all generated nodes.
- Its performance **depends heavily on the quality of the heuristic function**.

Example:

In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the

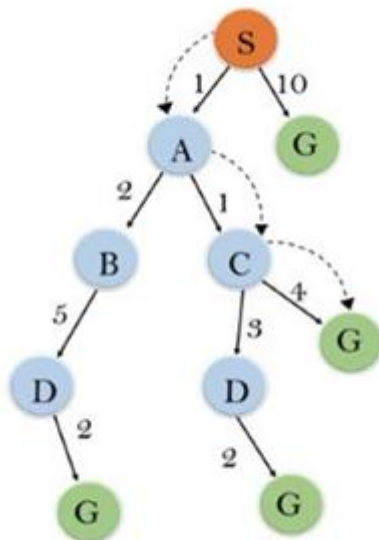
$f(n)$ of each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



1. Initialization Notation

- $OPEN = \{(S, f=5)\}$
- $CLOSED = \{\}$

2. Iteration Representation

Iteration 1 (after expanding S):

OPEN = {
(S→A, 4),
(S→G, 10)
}

Iteration 2 (after expanding A):

OPEN = {
(S→A→C, 4),
(S→A→B, 7),
(S→G, 10)
}

Iteration 3

Correct order:

OPEN = {
(S→A→C→G, 6),
(S→A→B, 7),
(S→G, 10),
(S→A→C→D, 11)
}

3. Iteration 4

A* stops immediately when the goal node is selected (not after another full iteration)

Using A* search, the nodes are expanded based on the lowest $f(n)$ value. The goal node G is selected with $f(n)=6$, resulting in the optimal path $S \rightarrow A \rightarrow C \rightarrow G$ with total cost 6.”

Complete: A* algorithm is **complete** as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is **optimal** if it **follows** below **two conditions**:

- **Admissible:** the first condition requires for optimality is that $h(n)$ should be an admissible heuristic for A* tree search.
An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.
If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d . So, the time complexity is $O(b^d)$, where b is the branching factor.

Space Complexity: The space complexity of A* search algorithm is $O(b^d)$

Local Search Algorithms and Optimization Problems:

Introduction to Local Search

- Traditional uninformed and informed search algorithms explore the search space systematically.
- They store paths and track explored alternatives.
- When a goal is reached, the **solution is the path** from **start to goal**.

However, in many real-world problems:

- The path is irrelevant
- Only the final solution/state matters

This leads to a different approach:

→ Local Search Algorithms

What is Local Search?

- Local search algorithms work with a single current state (node).
- They move to neighbouring states iteratively.

- They do not store paths.

Focus:

- Improve the current state
- Reach a better (or optimal) solution

Local Search Algorithms:

Local search algorithms are **search techniques** that **operate on a single current state** and move to its neighbouring states in order **to find an optimal or near-optimal solution**, without maintaining paths.

They are especially useful for solving optimization problems **where the goal is to find the best state rather than the path** to it.

Key Characteristics

- Operate on one node at a time
- Move to neighbouring states
- Do not maintain search trees or paths

Advantages of Local Search

1. Low Memory Usage
 - Uses constant or very little memory
2. Efficient for Large Spaces
 - Works well for:
 - Very large
 - Infinite
 - Continuous state spaces

In addition to finding goals, local search algorithms are useful for solving pure **optimization problems**, in which the aim is **to find the best state** according to an **objective function**.

To understand local search, we find it useful to consider the **state-space landscape** (as in Figure 4.1).

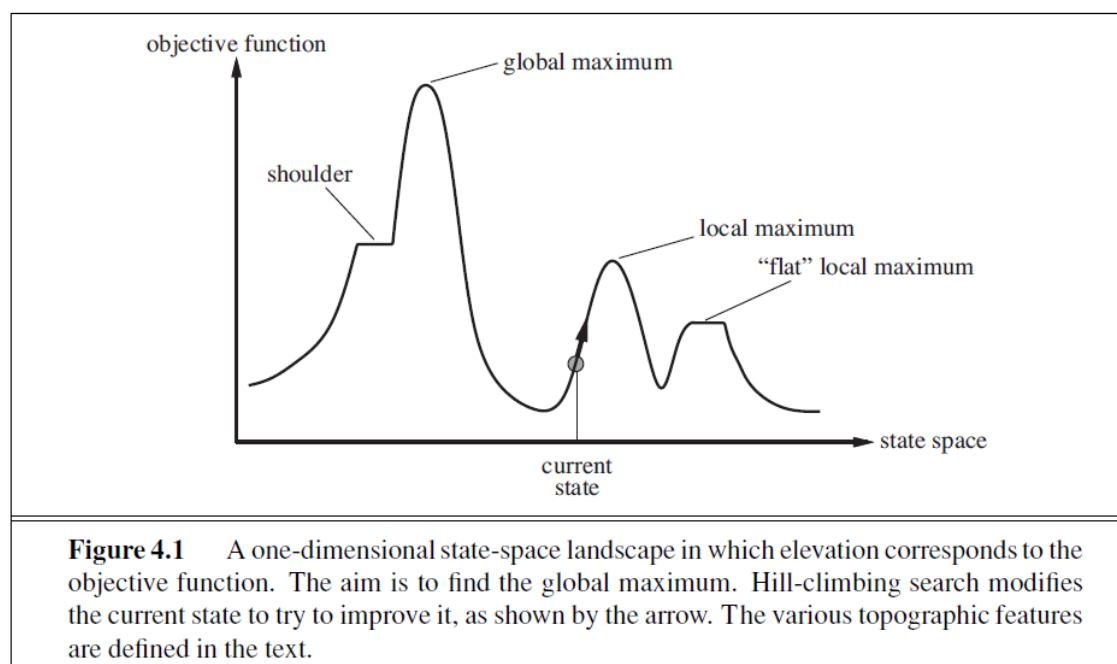
A **landscape** has both “**location**” (defined by the state) and “**elevation**” (defined by the value of the heuristic cost function or objective function).

If elevation corresponds to cost, then the aim is to find the lowest valley—a **global minimum**;

if elevation corresponds to an objective function, then the aim is to find the highest peak—a **global maximum**.

Local search algorithms explore this landscape.

A **complete local search algorithm** always finds a goal if one exists;
an **optimal algorithm** always finds a **global minimum/maximum**.



Hill Climbing Search (Steepest Ascent/Descent)

- At **each iteration**, the hill-climbing search algorithm **moves to the best successor** of the current node **according** to an **objective function**.
- Best successor is the successor **with best value** (highest or lowest) according to an objective function.
- If no successors have better value than the current value, it returns.
- It moves in direction of uphill (hill climbing).
- It **terminates** when it **reaches** a “**peak**” where no neighbour has a higher value.

- The algorithm **does not maintain a search tree**, so the data structure for the current node need only record the **state** and the **value of the objective function**.
- Hill climbing **does not look ahead beyond the immediate neighbours** of the current state.
- Hill climbing is **sometimes called greedy local search** because it grabs a good neighbour state without thinking ahead about where to go next.
- Greedy algorithms often perform quite well and
- Hill climbing often makes rapid progress toward a solution.

```

function HILL-CLIMBING(problem) returns a state that is a local maximum

  current  $\leftarrow$  MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor  $\leftarrow$  a highest-valued successor of current
    if neighbor.VALUE  $\leq$  current.VALUE then return current.STATE
    current  $\leftarrow$  neighbor

```

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor; in this version, that means the neighbor with the highest VALUE, but if a heuristic cost estimate h is used, we would find the neighbor with the lowest h .

```

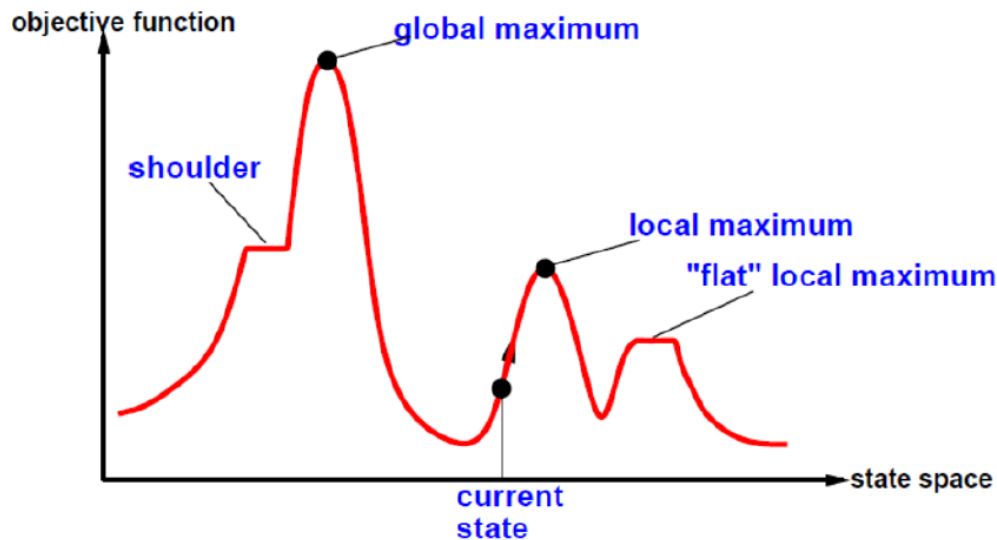
function HILL-CLIMBING(problem) returns a state that is a local maximum

  current  $\leftarrow$  MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor  $\leftarrow$  a highest-valued successor of current
    if neighbor.VALUE  $\leq$  current.VALUE then return current.STATE
    current  $\leftarrow$  neighbor

```

best successor

- The aim is to find the highest peak -a **global maximum**.
- Hill-climbing search modifies the current state to try to improve it.



A **local maximum** is a peak that is **higher than each of its neighbouring states** but lower than the **global maximum**.

- Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upward toward the peak but will then be stuck with nowhere else to go.

- A **plateau** is a **flat area** of the state-space landscape. It can be a **flat local maximum**, from which no uphill exit exists, or a **shoulder**, from which progress is possible.

- A hill-climbing search might get lost on the plateau.

- Random sideways moves** can escape from *shoulders* but they loop forever on *flat maxima*.

- Flat maxima → A completely flat region
- No higher state exists anywhere nearby
- Taking **random sideways moves** can help you **get out of slightly flat regions**, but if the **top is completely flat**, you may end **up moving around endlessly** without making progress.

Travelling Salesman Problem (for Hill Climbing Explanation)

Consider 5 cities with distances as follows:

City	A	B	C	D	E
A	0	2	9	10	7

City	A	B	C	D	E
B	2	0	6	4	3
C	9	6	0	8	5
D	10	4	8	0	6
E	7	3	5	6	0

Goal: Find the **shortest route** visiting all cities once and returning to start.

Solution -

Step 1: Initial Solution

Start with a random route:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$

Calculate cost:

$A \rightarrow B = 2$

$B \rightarrow C = 6$

$C \rightarrow D = 8$

$D \rightarrow E = 6$

$E \rightarrow A = 7$

Total = 29

Step 2: Generate Neighbours

Create neighbours by swapping cities:

Examples:

- $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow A$
- $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow A$
- $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow A$

Step 3: Evaluate Neighbours

Example:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow A$

$A \rightarrow B = 2$

$B \rightarrow D = 4$

$D \rightarrow C = 8$

$C \rightarrow E = 5$

$E \rightarrow A = 7$

Total = 26

Better than 29

Step 4: Move to Better Neighbour

New route:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow A$ (cost = 26)

Step 5: Repeat

Check neighbours again and move to better ones.

Step 6: Termination

Stop when:

- No neighbour has lower cost

This is a **local optimum (best nearby solution)**